

系统开发工具基础 · 实验报告

第 2 周：命令行环境 · 开发环境与工具 · Debugging and Profiling

姓 名 徐宜安

学 号 25060012033

授课教师 周小伟

上课日期 2026 年 8 月 31 日

Git 仓库 [GitHub 课程仓库](#)

报告日期 2026 年 9 月 3 日

1 课上练习

1.1 q05 · 控制一个可清理的后台任务（进程、信号与任务控制）

题目要求

将给定脚本保存为 `worker.sh`，并完成以下操作：

- 赋予脚本执行权限，在前台启动，将 `stdout`、`stderr` 分别写入两个日志文件；
- 使用 `Ctrl-Z` 挂起任务，再通过 `bg` 放到后台，使用 `jobs` 确认状态；
- 用 `jobs -p` 取得 `PID`，发送 `SIGTERM` 并等待任务结束；
- 确认 `cleanup.log` 中出现 `CLEAN_EXIT`，且任务已经退出。

```
1  #!/usr/bin/env bash
2  trap 'echo CLEAN_EXIT >> cleanup.log; exit 0' TERM INT
3  n=0
4  while true; do
5      echo "$n"
6      n=$((n+1))
7      sleep 1
8  done
```

代码 1 `worker.sh`：带信号清理逻辑的后台任务

实验过程：先用 `chmod +x worker.sh` 增加执行权限，随后运行 `./worker.sh`，并分别用 `1>stdout.log` 和 `2>stderr.log` 重定向标准输出、标准错误。运行后按 `Ctrl-Z` 后任务进入 `Stopped` 状态，再执行 `bg`，`jobs` 显示它已在后台继续运行。最后使用 `kill $(jobs -p)` 直接取得后台任务 `PID` 并发送 `SIGTERM`。完整操作见图 1。

实验结果：收到 `SIGTERM` 后，脚本执行 `trap` 中的清理命令并正常退出，终端显示任务状态为 `Done`，`cleanup.log` 中写入了 `CLEAN_EXIT`。`stdout.log` 保存了 0 到 12（与任务的运行时间有关），`stderr.log` 为空，说明标准输出和标准错误已正确分开。

```
xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础/第二周/课上/q05 (main)
$ chmod +x worker.sh

xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础/第二周/课上/q05 (main)
$ ./worker.sh 1>stdout.log 2>stderr.log

[1]+  Stopped                  ./worker.sh > stdout.log 2> stderr.log

xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础/第二周/课上/q05 (main)
$ bg
[1]+  ./worker.sh > stdout.log 2> stderr.log &

xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础/第二周/课上/q05 (main)
$ jobs
[1]+  Running                  ./worker.sh > stdout.log 2> stderr.log &

xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础/第二周/课上/q05 (main)
$ kill $(jobs -p)

xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础/第二周/课上/q05 (main)
$ cat cleanup.log
[1]+  Done                    ./worker.sh > stdout.log 2> stderr.log
CLEAN_EXIT
```

图 1 q05 任务挂起、后台恢复、发送信号与清理结果

1.2 q06 · 语义重构与本地开发反馈（开发环境与工具）

题目要求

在 q06 中创建给定的三个 Python 文件，并完成以下操作：

- 配置 Python、pytest、ruff 和 Python 语言服务器；
- 演示跳转到定义和查找引用；
- 使用“重命名符号”把 `total_price` 改为 `calculate_total`，同步修改定义和全部引用；
- 临时加入未使用的 `import os`，用 ruff 找出并删除，最后保证 ruff 和 pytest 均通过。

实验过程：首先确认 Python、pytest、ruff 的版本，并检查 VS Code 中的 Python、Pylance 和 Python Debugger 插件，环境配置见图 2。

在 `app.py` 中按住 Ctrl 点击 `total_price`，编辑器能够定位到 `math_utils.py` 中的函数定义（图 3）。随后鼠标右键点击 `total_price` 并在菜单中选择“查找所有引用”，VS Code 在三个文件中找到 5 处结果，包括函数定义、两处导入和两处调用，过程见图 4。

接着对函数右键选择“重命名符号”，输入新名称 `calculate_total`。这一步由语言服务器统一更新符号，没有替换 `test_total_price`，操作见图 5。

重命名后的三个文件如下。

```
1 # math_utils.py
2 def calculate_total(price: float, count: int) -> float:
3     return price * count
```

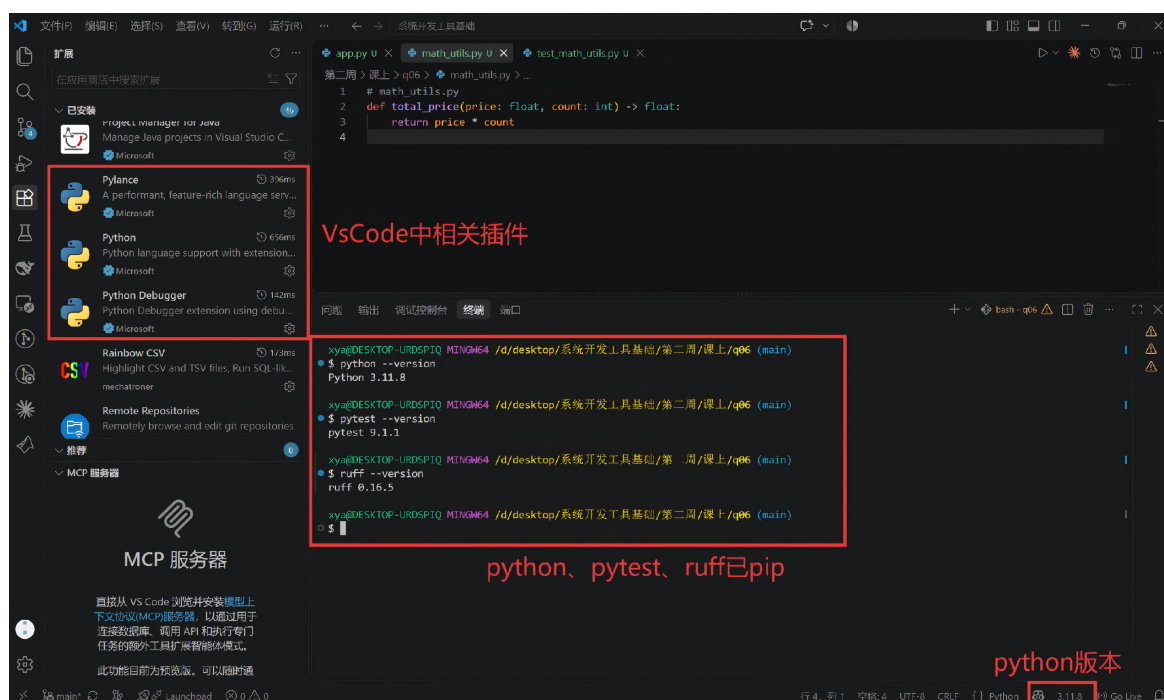


图 2 q06 Python 工具版本与 VS Code 插件配置

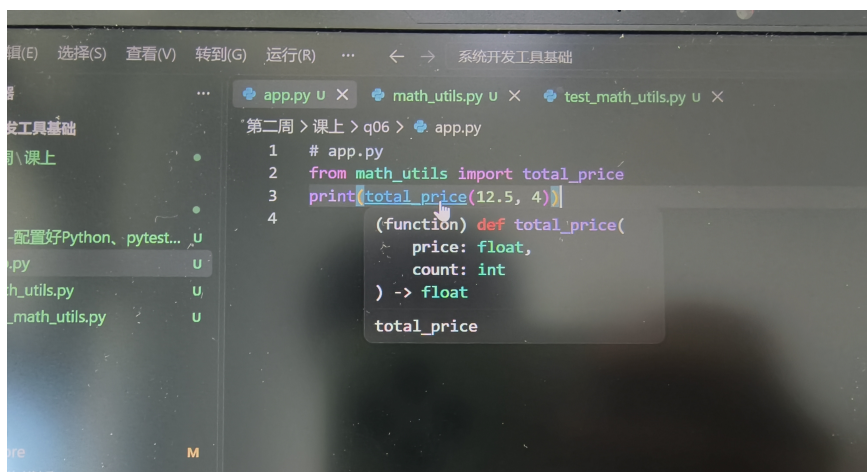


图 3 按住 Ctrl 点击函数调用并跳转到定义

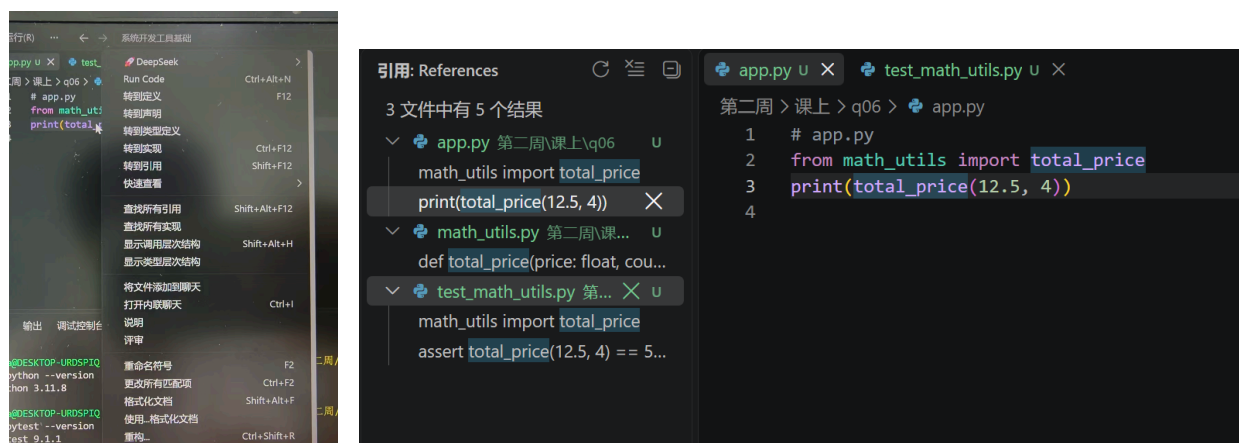


图 4 从右键菜单查找引用，并列出三个文件中的 5 处结果

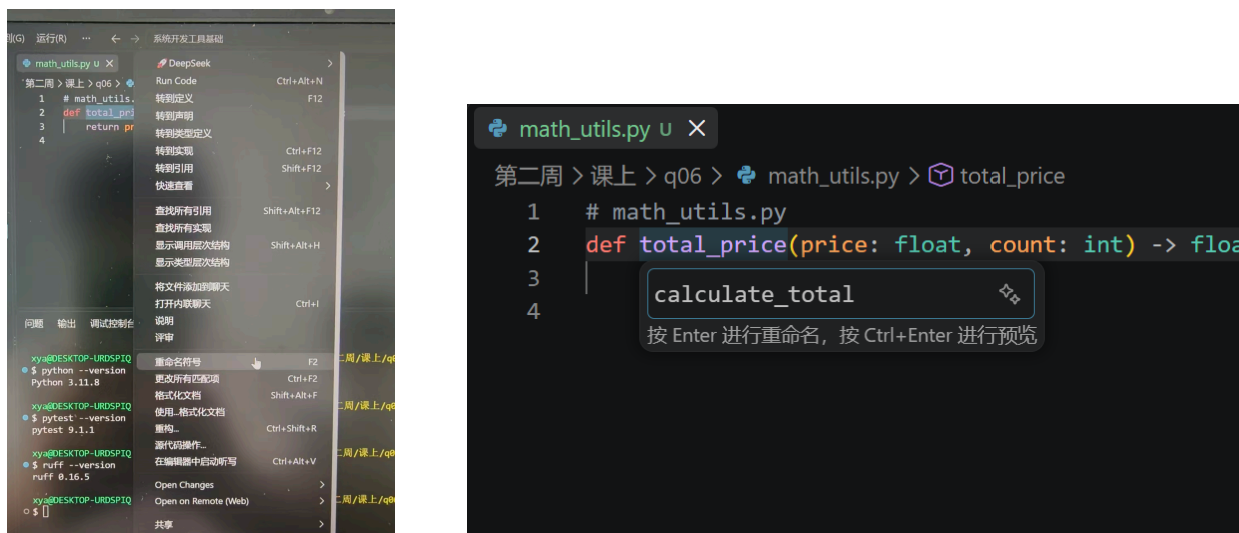


图 5 使用“重命名符号”将函数名改为 calculate_total

代码 2 math_utils.py: 重命名后的函数定义

```
1 # app.py
2 from math_utils import calculate_total
3
4 print(calculate_total(12.5, 4))
```

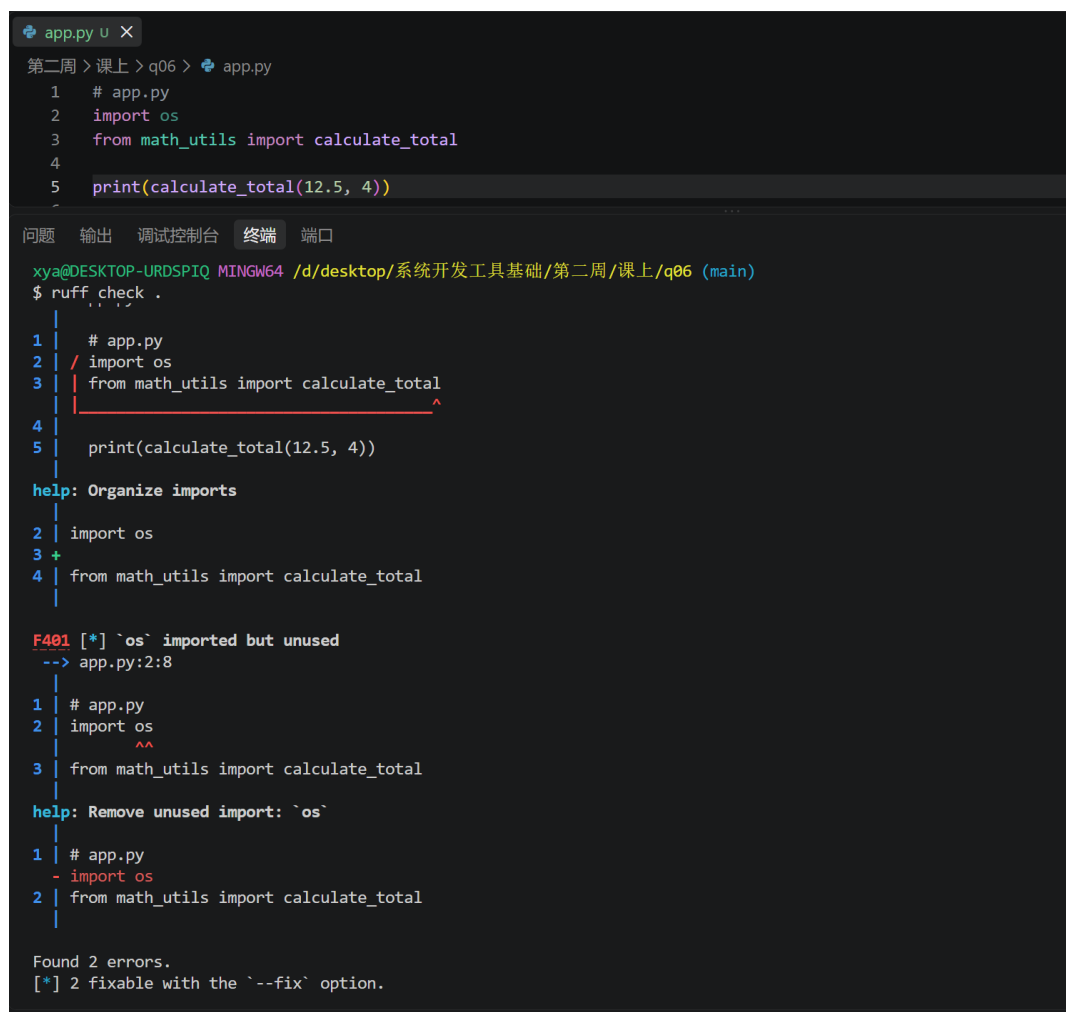
代码 3 app.py: 重命名后的导入与调用

```
1 # test_math_utils.py
2 from math_utils import calculate_total
3
4
5 def test_total_price():
6     assert calculate_total(12.5, 4) == 50.0
```

代码 4 test_math_utils.py: 同步更新后的测试

最后在 app.py 中临时加入 import os。第一次执行 ruff check . 时, ruff 报告导入顺序问题和未使用的 os, 并指出 F401 的具体位置 (图 6)。用 ruff check . --fix 修复相关问题, 再重新运行检查和测试。

实验结果: 最终 ruff check . 输出 All checks passed!, pytest 收集并通过 1 个测试, 耗时 0.02 s (图 7)。函数改名后仍能正确计算 $12.5 \times 4 = 50.0$, 说明语义重构没有破坏原有功能。



```
app.py u X
第二周 > 课上 > q06 > app.py
1 # app.py
2 import os
3 from math_utils import calculate_total
4
5 print(calculate_total(12.5, 4))

问题 输出 调试控制台 终端 端口
xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第二周/课上/q06 (main)
$ ruff check .
1 | # app.py
2 | / import os
3 | | from math_utils import calculate_total
4 | | ^
5 | | print(calculate_total(12.5, 4))
6 | |
help: Organize imports
2 | import os
3 +
4 | from math_utils import calculate_total
5 |
F401 [*] `os` imported but unused
--> app.py:2:8
1 | # app.py
2 | import os
3 | ^
4 |
help: Remove unused import: `os`
1 | # app.py
- import os
2 | from math_utils import calculate_total
3 |

Found 2 errors.
[*] 2 fixable with the `--fix` option.
```

图 6 ruff 检出未使用的 os 和导入顺序问题



```
xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第二周/课上/q06 (main)
$ ruff check . --fix
Found 1 error (1 fixed, 0 remaining).

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第二周/课上/q06 (main)
$ ruff check .
All checks passed!

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第二周/课上/q06 (main)
$ pytest test_math_utils.py -v
===== test session starts =====
platform win32 -- Python 3.11.8, pytest-9.1.1, pluggy-1.6.0 -- D:\Software\python.exe
cachedir: .pytest_cache
rootdir: D:\desktop\系统开发工具基础\第二周\课上\q06
plugins: anyio-4.12.0
collected 1 item

test_math_utils.py::test_total_price PASSED [100%]

===== 1 passed in 0.02s =====
```

图 7 ruff 自动修复后再次检查，并运行 pytest 验证

1.3 q07 · 用调试器定位归并排序缺陷 (Debugging)

题目要求

运行给定的归并排序代码，使用调试器定位并修复缺陷：

- 先运行给定输入，确认结果错误或出现异常；
- 在 `merge` 中设置断点，观察 `i`、`j`、`left[i]` 和 `right[j]`；
- 完成最小修复，不得使用 `sorted` 替换归并排序；
- 增加给定输入和含重复元素列表两个 `pytest` 测试，并保证测试通过。

实验过程：原程序运行后得到 `[1, 5, 4, 3, 4, 5, 6, 9]`，结果没有按升序排列，修复前的代码和输出见图 10(a)。

在 `merge` 的比较语句处设置断点，将 `left[i]` 和 `right[j]` 加入监视窗口。调试开始时 `i=j=0`。此时 `left=[3]`、`right=[1]`，监视值分别为 3 和 1（图 8）。继续执行到 `left=[1,3]`、`right=[1,4]`、`i=j=1` 时，程序进入 `else` 分支，却读取了 `right[i]`。此时左右列表下标刚好相同，所以没有立即报错，但代码使用的下标与递增的 `j` 不一致，问题位置见图 9。

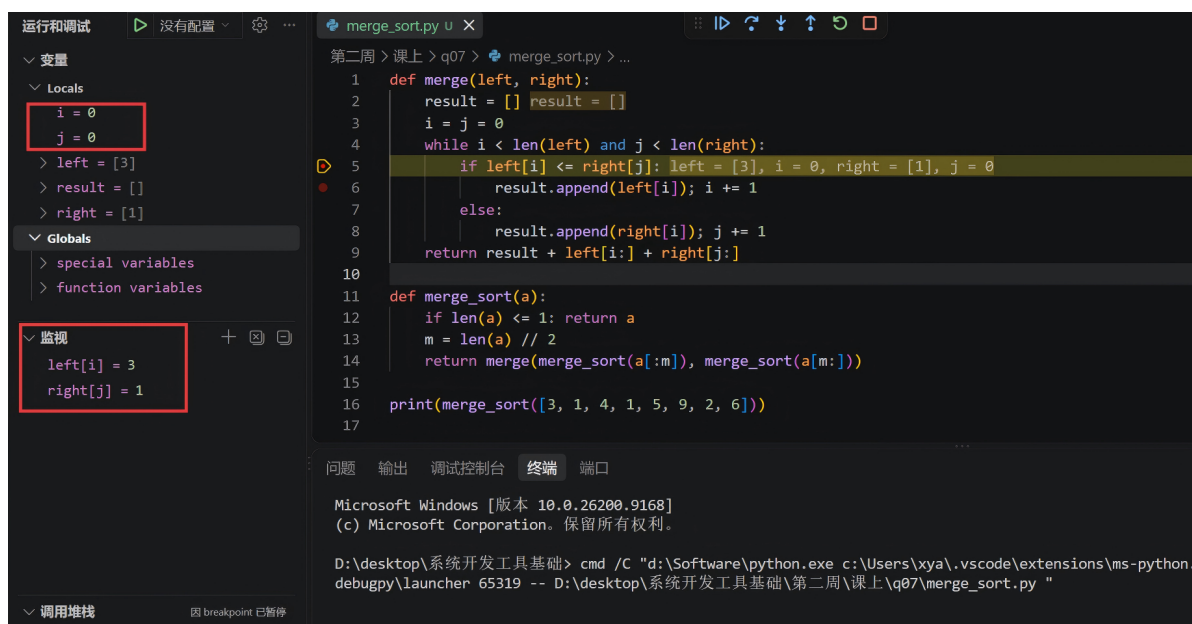


图 8 在 `merge` 中设置断点并监视左右列表及其下标

最小修复是把第 8 行的 `right[i]` 改成 `right[j]`，其余归并排序逻辑保持不变。最终代码见代码 5。修复后再次运行同一输入，输出变为 `[1,1,2,3,4,5,6,9]`，见图 10(b)。

```
1 def merge(left, right):
2     result = []
3     i = j = 0
4     while i < len(left) and j < len(right):
5         if left[i] <= right[j]:
6             result.append(left[i]); i += 1
```

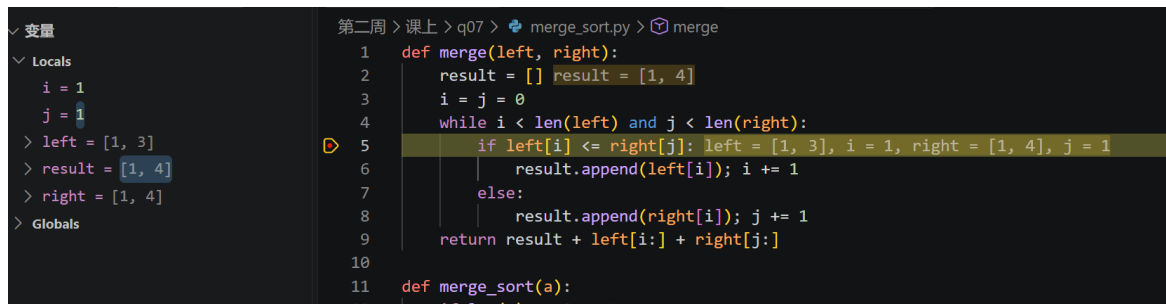


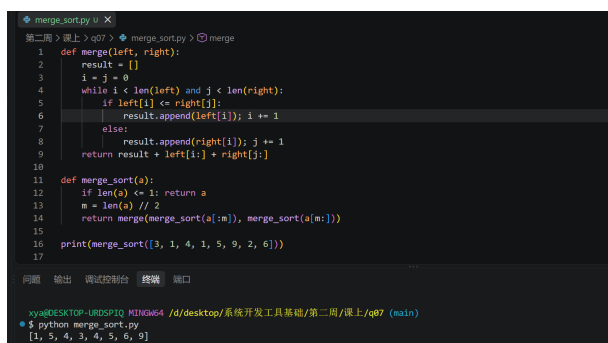
图 9 调试过程中定位到 else 分支使用了错误下标 i

```

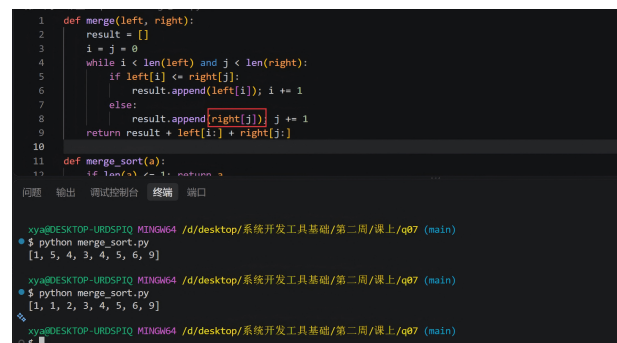
7         else:
8             result.append(right[j]); j += 1
9         return result + left[i:] + right[j:]
10
11 def merge_sort(a):
12     if len(a) <= 1: return a
13     m = len(a) // 2
14     return merge(merge_sort(a[:m]), merge_sort(a[m:]))
15
16 print(merge_sort([3, 1, 4, 1, 5, 9, 2, 6]))

```

代码 5 merge_sort.py: 修复下标后的归并排序



(a) 修复前: 输出没有正确排序



(b) 修复后: 同一输入得到升序结果

图 10 q07 修复前后的代码运行结果对比

随后在 test_merge.py 中加入两个测试, 分别覆盖题目给定输入和包含多个重复元素的列表。

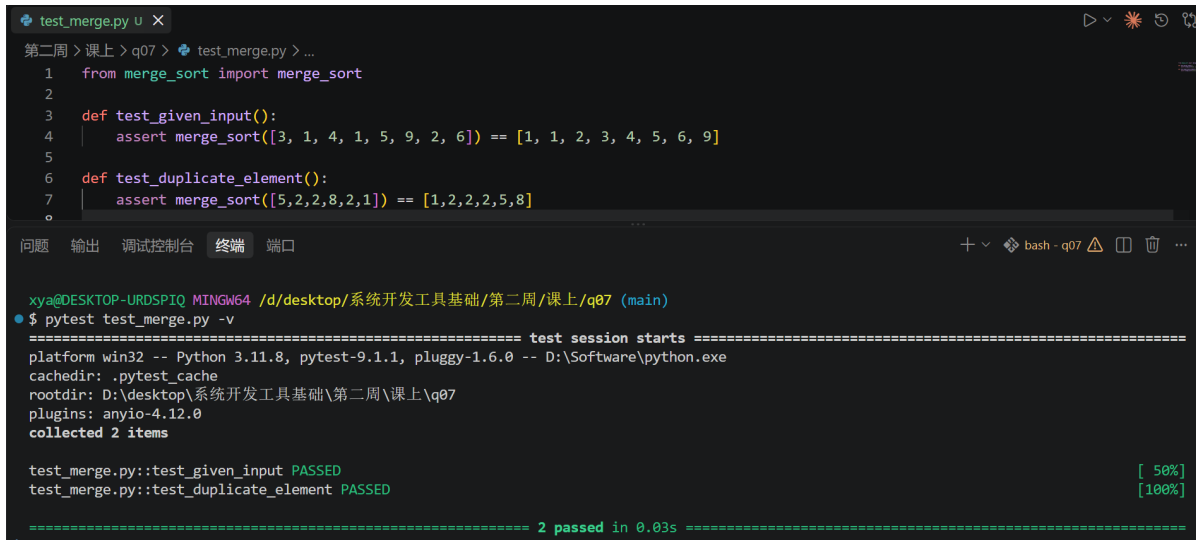
```

1 from merge_sort import merge_sort
2
3 def test_given_input():
4     assert merge_sort([3, 1, 4, 1, 5, 9, 2, 6]) == [1, 1, 2, 3, 4, 5, 6, 9]
5
6 def test_duplicate_element():

```

```
7      assert merge_sort([5,2,2,8,2,1]) == [1,2,2,2,5,8]
```

代码 6 test_merge.py: 给定输入与重复元素测试



```
test_merge.py u X
第二周 > 课上 > q07 > test_merge.py > ...
1  from merge_sort import merge_sort
2
3  def test_given_input():
4      assert merge_sort([3, 1, 4, 1, 5, 9, 2, 6]) == [1, 1, 2, 3, 4, 5, 6, 9]
5
6  def test_duplicate_element():
7      assert merge_sort([5,2,2,8,2,1]) == [1,2,2,2,5,8]
8
9
问题 输出 调试控制台 终端 端口
+ v bash - q07 [ 50% ] [ 100% ]

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第二周/课上/q07 (main)
$ pytest test_merge.py -v
===== test session starts =====
platform win32 -- Python 3.11.8, pytest-9.1.1, pluggy-1.6.0 -- D:\Software\python.exe
cachedir: .pytest_cache
rootdir: D:\desktop\系统开发工具基础\第二周\课上\q07
plugins: anyio-4.12.0
collected 2 items

test_merge.py::test_given_input PASSED
test_merge.py::test_duplicate_element PASSED

===== 2 passed in 0.03s =====
```

图 11 两个归并排序测试均通过

实验结果：pytest 共收集 2 个测试，给定输入和重复元素用例全部通过，耗时 0.03 s (图 11)。

1.4 q08 · 先测量，再优化慢速词频程序 (Profiling)

题目要求

生成固定的 30000 个单词输入，测量并优化低效去重程序：

- 使用 time 运行原程序 2 次并记录耗时；
- 使用 cProfile -s cumulative 运行一次，找出主要耗时位置；
- 在不改变最终 count 的前提下优化程序，且不得写死 1000；
- 使用相同输入再运行 2 次，计算优化前后中位数和加速比。

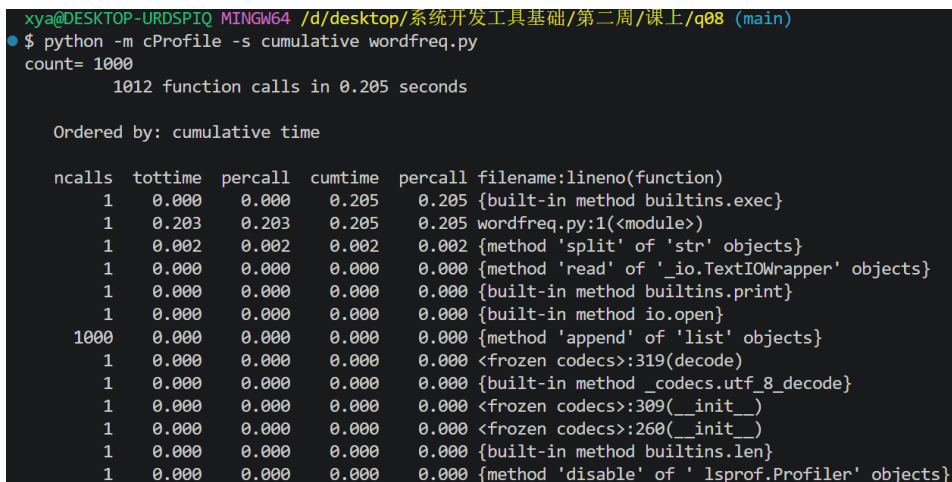
实验过程：先运行 generate_words.py，生成可重复使用的 words.txt。

```
1 # generate_words.py
2 import random
3
4 random.seed(20260831)
5 vocab = [f"w{i}" for i in range(1000)]
6 with open("words.txt", "w", encoding="utf-8") as f:
7     f.write(" ".join(random.choice(vocab) for _ in range(30000)))
```

代码 7 generate_words.py: 生成固定随机词表

使用 `time` 命令运行程序，两次 `real` 时间分别为 0.423 s 和 0.398 s，输出计数均为 1000，见图 13(a)。

`python -m cProfile -s cumulative wordfreq.py` 显示程序共运行 0.205 s，其中顶层模块 `wordfreq.py:1(<module>)` 的累计时间为 0.203 s，文件读取和 `split` 只占 0.002 s（图 12）。耗时主要集中在模块内反复执行的列表成员查找，而不是读取输入文件。



```

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第二周/课上/q08 (main)
$ python -m cProfile -s cumulative wordfreq.py
count= 1000
1012 function calls in 0.205 seconds

Ordered by: cumulative time

ncalls  tottime  percall  cumtime  percall filename:lineno(function)
1      0.000    0.000    0.205    0.205 {built-in method builtins.exec}
1      0.203    0.203    0.205    0.205 wordfreq.py:1(<module>)
1      0.002    0.002    0.002    0.002 {method 'split' of 'str' objects}
1      0.000    0.000    0.000    0.000 {method 'read' of '_io.TextIOWrapper' objects}
1      0.000    0.000    0.000    0.000 {built-in method builtins.print}
1      0.000    0.000    0.000    0.000 {built-in method io.open}
1000    0.000    0.000    0.000    0.000 {method 'append' of 'list' objects}
1      0.000    0.000    0.000    0.000 <frozen codecs>:319(decode)
1      0.000    0.000    0.000    0.000 {built-in method _codecs.utf_8_decode}
1      0.000    0.000    0.000    0.000 <frozen codecs>:309(__init__)
1      0.000    0.000    0.000    0.000 <frozen codecs>:260(__init__)
1      0.000    0.000    0.000    0.000 {built-in method builtins.len}
1      0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}

```

图 12 cProfile 按累计时间排序的分析结果

将 `unique` 改成集合后，每个单词直接使用 `add` 加入集合，优化后的完整程序见代码 8。

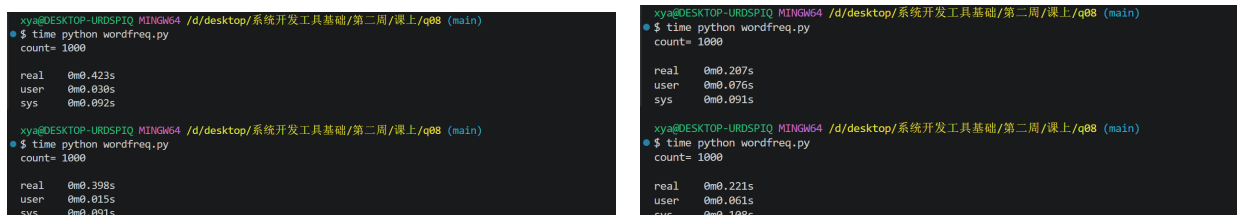
```

1 # wordfreq.py
2 words = open("words.txt", encoding="utf-8").read().split()
3 unique = set()
4 for word in words:
5     unique.add(word)
6 print("count=", len(unique))

```

代码 8 wordfreq.py：使用集合完成去重

使用同一个 `words.txt` 再运行两次，`real` 时间分别为 0.207 s 和 0.221 s，两次输出的计数仍为 1000，见图 13(b)。



```

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第二周/课上/q08 (main)
$ time python wordfreq.py
count= 1000

real    0m0.423s
user    0m0.030s
sys     0m0.092s

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第二周/课上/q08 (main)
$ time python wordfreq.py
count= 1000

real    0m0.398s
user    0m0.015s
sys     0m0.091s

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第二周/课上/q08 (main)
$ time python wordfreq.py
count= 1000

real    0m0.207s
user    0m0.076s
sys     0m0.091s

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第二周/课上/q08 (main)
$ time python wordfreq.py
count= 1000

real    0m0.221s
user    0m0.061s
sys     0m0.100s

```

(a) 原程序两次运行时间

(b) 集合优化后的两次运行时间

图 13 q08 使用相同输入优化前后的计时结果

表 1 q08 优化前后运行时间与加速比

版本	第 1 次 / s	第 2 次 / s	中位数 / s
列表去重	0.423	0.398	0.4105
集合去重	0.207	0.221	0.2140

实验结果:两次测量的中位数分别为 $(0.423+0.398)/2 = 0.4105$ s 和 $(0.207+0.221)/2 = 0.2140$ s, 加速比为 $0.4105/0.2140 \approx 1.92$ 。优化后运行时间约为原来的 52%, 同时计数仍为 1000。完整计时对比见表 1。

2 课后练习

参考资料记录 [Missing Semester: 命令行环境](#)、[Missing Semester: 开发环境与工具](#)、[Missing Semester: 编辑器 \(Vim\)](#)、[Missing Semester: Debugging and Profiling](#)。

2.1 命令行环境

1. **多参数与花括号展开。**使用 `mkdir -p demo/{src,docs,logs}` 一次建立三个子目录, 再用 `touch demo/src/{main,utils,test}.py` 创建三个源码文件; `find` 输出的目录树说明两次展开都符合预期, 见图 14。

```
xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第二周/课后/command-line (main)
• $ mkdir -p demo/{src,docs,logs}; touch demo/src/{main,utils,test}.py

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第二周/课后/command-line (main)
• $ find demo -maxdepth 2 -print
demo
demo/docs
demo/logs
demo/src
demo/src/main.py
demo/src/test.py
demo/src/utils.py
```

图 14 花括号展开创建练习目录与文件

2. **通配符展开。**将 `demo/src/*.py` 交给 Shell 展开, `printf` 依次输出 `main.py`、`test.py` 和 `utils.py`, 结果见图 15。

```
xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第二周/课后/command-line (main)
• $ printf '%s\n' demo/src/*.py
demo/src/main.py
demo/src/test.py
demo/src/utils.py
```

图 15 通配符匹配三个 Python 文件

3. **分离标准输出与标准错误。**用文件描述符 1、2 分开输出: 普通消息写入 `stdout.log`, 警告消息写入 `stderr.log`; 随后读取文件确认两路输出没有混在一起, 见图 16。

```
xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第二周/课后/command-line (main)
$ bash -c 'echo "normal message"; echo "warning message" >&2' >demo/logs/stdout.log 2>demo/logs/stderr.log
echo "stdout: "; cat demo/logs/stdout.log
echo "stderr: "; cat demo/logs/stderr.log
stdout:
normal message
stderr:
warning message
```

图 16 标准输出与标准错误分别写入日志

4. 使用管道处理数据流。把消息依次送入 `sort`、`uniq -c` 和 `sort -rn`，再由 `tee` 同时显示并保存统计结果，最终 `error` 出现 3 次，见图 17。

```
xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第二周/课后/command-line (main)
$ printf 'error\nok\nerror\nwarning\nerror\n' | sort | uniq -c | sort -nr | tee demo/logs/counts.txt
3 error
1 warning
1 ok
```

图 17 管道完成消息频次统计

5. 比较单引号与双引号。对变量 `tool` 分别使用双引号和单引号，前者输出变量值 `ruff`，后者保留字面量 `$tool`，差别见图 18。

```
xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第二周/课后/command-line (main)
$ tool=ruff

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第二周/课后/command-line (main)
$ printf 'double=%s\nsingle=%s\n' "$tool" '$tool'
double=ruff
single=$tool
```

图 18 单引号与双引号的变量展开差异

6. 命令替换。在 `$(...)` 中组合 `find` 和 `wc -l`，将统计值保存到 `py_count`；输出 `python files=3` 与实际文件数一致，见图 19。

```
xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第二周/课后/command-line (main)
$ py_count=$(find demo/src -name '*.py' | wc -l)

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第二周/课后/command-line (main)
$ echo "python files=$py_count"
python files=3
```

图 19 命令替换获取 Python 文件数量

7. 进程替换。两个 `<(printf ...)` 分别产生一份临时输入供 `diff` 比较，输出指出第二行分别是 `utils.py` 和 `test.py`；此时 `diff` 返回非零值只是表示发现差异，比较过程本身没有失败，见图 20。

8. 临时变量与导出变量。`COURSE_MODE` 只对紧随其后的命令生效，父 Shell 中仍为 `unset`；经 `export` 设置的 `COURSE_WEEK` 能被子 Shell 读取，并在 `unset` 后消失，完整结果见图 21。

9. 返回码与短路执行。`grep` 找到 `error` 后执行 `&&` 右侧命令，找不到 `fatal` 时执行 `||` 右侧命令；随后用 `$?` 取得 `false` 的返回码 1，见图 22。

```
xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础/第二周/课后/command-line (main)
❶ $ diff <(printf 'main.py\nutils.py\n') <(printf 'main.py\ntest.py\n')
2c2
< utils.py
---
> test.py
```

图 20 进程替换生成两路 diff 输入

```
xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础/第二周/课后/command-line (main)
$ COURSE_MODE=temporary bash -c 'echo "one command sees COURSE_MODE=$COURSE_MODE"'
echo "parent sees COURSE_MODE=${COURSE_MODE:-unset}"
export COURSE_WEEK=2
bash -c 'echo "child sees COURSE_WEEK=$COURSE_WEEK"'
unset COURSE_WEEK
❷ echo "after unset=${COURSE_WEEK:-unset}"
one command sees COURSE_MODE=temporary
parent sees COURSE_MODE=unset
child sees COURSE_WEEK=2
after unset=unset
```

图 21 环境变量的作用范围与继承

```
xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础/第二周/课后/command-line (main)
$ grep -q error demo/logs/counts.txt && echo "found error"
grep -q fatal demo/logs/counts.txt || echo "fatal not found"
false
❸ echo "false exit code=$?"
found error
fatal not found
false exit code=1
```

图 22 条件执行与返回码检查

10. 获取后台 PID 并发送信号。后台启动 sleep 后用 \$! 保存其 PID, 再发送 SIGTERM 并 wait; 返回码 143 等于 128 + 15, 说明进程确实由 15 号信号终止, 见图 23。

```
xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础/第二周/课后/command-line (main)
$ sleep 30 &
pid=$!
echo "background pid=$pid"
kill -TERM "$pid"
wait "$pid"
❹ echo "wait exit code=$?"
[1] 927
background pid=927
[1]+  Terminated                  sleep 30
wait exit code=143
```

图 23 向后台进程发送 SIGTERM

11. 查看命令程序的信息。依次使用 --version、--help 和 type -a 查看 Python 版本、常用选项及系统中全部同名可执行文件位置, 结果见图 24。

```
xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第二周/课后/command-line (main)
$ python --version
python --help | head -n 8
• type -a python
Python 3.11.8
usage: D:\Software\python.exe [option] ... [-c cmd | -m mod | file | -] [arg] ...
Options (and corresponding environment variables):
-b      : issue warnings about str(bytes_instance), str(bytearray_instance)
         and comparing bytes/bytearray with str. (-bb: issue errors)
-B      : don't write .pyc files on import; also PYTHONDONTWRITEBYTECODE=x
-c cmd  : program passed in as string (terminates option list)
-d      : turn on parser debugging output (for experts only, only works on
         debug builds); also PYTHONDEBUG=x
python is /d/Software/python
python is /d/Software/ucrt64/bin/python
python is /c/Users/xya/AppData/Local/Microsoft/WindowsApps/python
python is /d/Software/ucrt64/bin/python
```

图 24 Python 的版本、帮助与程序位置

2.2 开发环境与工具

前七项在 VS Code 中观察语言服务器和扩展提供的开发功能，后三项改用 Vim 完成同一个 FizzBuzz 文件的录入和修改，分别体验图形化 IDE 与终端编辑器的工作方式。

1. 查看 VS Code 与现有扩展。通过 `code --version` 查看编辑器版本，再筛选扩展列表，确认 Python、debugpy 和 Pylance 已经安装，见图 25。

```
xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第二周/课后/development (main)
$ code --version
• code --list-extensions | grep -Ei "python|pylance|ruff"
1.135.0
08d4889f9ec4a1685d257b9b95de036c8e1ce1e5
x64
ms-python.autopep8
ms-python.debugpy
ms-python.python
ms-python.vscode-pylance
```

图 25 VS Code 版本及 Python 开发扩展

2. 从命令行安装开发扩展。使用 `code --install-extension` 安装 Ruff 和 Error Lens，并再次查询扩展列表，命令输出表明二者均安装成功。

```
xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第二周/课后/development (main)
$ code --install-extension charliermarsh.ruff
code --install-extension usernamehw.errorlens
• code --list-extensions | grep -Ei "ruff|errorlens"
Installing extensions...
(node:31152) [DEP0169] DeprecationWarning: `url.parse()` behavior is not standardized and prone to errors that have security implications. Use the WHATWG URL API instead. CVEs are not issued for `url.parse()` vulnerabilities.
(Use `Code --trace-deprecation ...` to show where the warning was created)
Installing extension 'charliermarsh.ruff'...
Extension 'charliermarsh.ruff' v2026.76.0 was successfully installed.
Installing extensions...
(node:53512) [DEP0169] DeprecationWarning: `url.parse()` behavior is not standardized and prone to errors that have security implications. Use the WHATWG URL API instead. CVEs are not issued for `url.parse()` vulnerabilities.
(Use `Code --trace-deprecation ...` to show where the warning was created)
Installing extension 'usernamehw.errorlens'...
Extension 'usernamehw.errorlens' v3.28.0 was successfully installed.
charliermarsh.ruff
usernamehw.errorlens
```

图 26 从命令行安装并确认开发扩展

3. 选择并验证 Python 解释器。在 VS Code 中按 `Ctrl+Shift+P` 打开命令面板，通过

Python: Select Interpreter 选择解释器，再在终端打印版本和路径；最终使用 Python 3.11.8，位置为 D:/Software/python.exe，见图 27。

```
xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第二周/课后/development (main)
$ python -c "import sys; print(sys.version); print(sys.executable)"
3.11.8 (tags/v3.11.8:db85d51, Feb 6 2024, 22:03:32) [MSC v.1937 64 bit (AMD64)]
D:\Software\python.exe
```

图 27 终端验证 VS Code 选用的 Python 解释器

下面几个实例使用带类型注解的 Product 类和 total 函数，基础代码见代码 9。

```
1 from dataclasses import dataclass
2
3
4 @dataclass
5 class Product:
6     name: str
7     price: float
8
9
10 def total(items: list[Product]) -> float:
11     return sum(item.price for item in items)
```

代码 9 Product 数据类与总价函数

4. 语义代码补全。在 Product 对象 first 后输入点号，Pylance 根据类定义给出 name、price 等成员候选，不需要手工记忆属性名，见图 28。

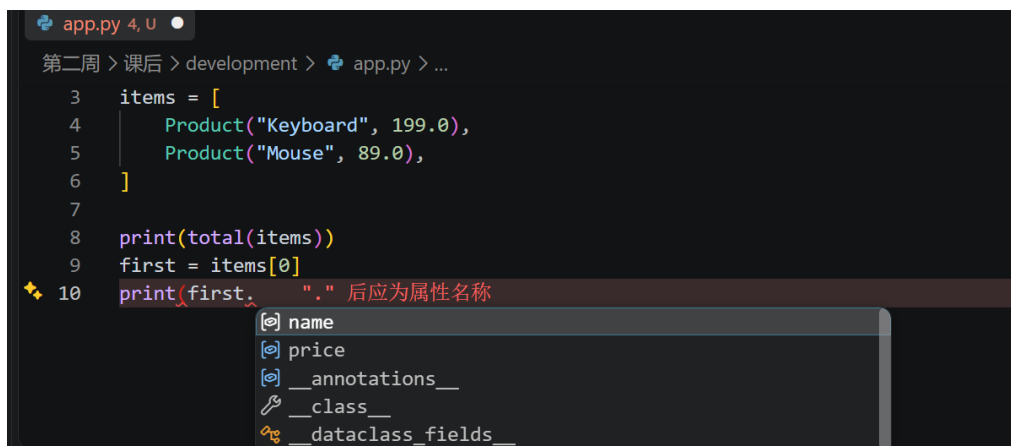


图 28 Pylance 根据对象类型补全成员

5. 查看函数签名。将光标停在 total(items) 调用处，悬停信息显示参数为 list[Product]、返回值为 float，见图 29。

6. 调用 Quick Fix 修复缺失导入。直接使用 datetime.now() 时，Pylance 标出未定义名称；按提示调出 Quick Fix 后出现自动补充导入的修复入口，过程见图 30。

```
6 ]
7     (function) def total(items: list[Product]) -> float
8     print(total(items))
9
```

图 29 查看 total 函数的类型签名

```
1 print(datetime.now())
未定义“datetime” Pylance(reportUndefinedVariable)
(function) datetime: Any
查看问题 (Alt+F8) 快速修复... (Ctrl+.) 修复 (Ctrl+I)
```

图 30 未定义名称对应的 Quick Fix 入口

7. 使用类型检查发现参数错误。把字符串 "39.9" 传给要求 float 的 price 参数, Pylance 在运行前便报告 reportArgumentType, 具体提示见图 31。

```
app.py 1, U X
第二周 > 课后 > development > app.py > ...
1 from product import Product, total
2
3 items = [
4     Product("Keyboard", 100),
5     Product("Mouse", 10),
6     Product("Cable", "39.9"),
7 ]
8
```

无法将“Literal['39.9']”类型的参数分配给函数“__init__”中类型为“float”的参数“price”
“Literal['39.9']”不可分配给“float” Pylance(reportArgumentType)

查看问题 (Alt+F8) 快速修复... (Ctrl+.) 修复 (Ctrl+I)

图 31 Pylance 报告参数类型不兼容

8. 使用 Vim 创建并保存文件。在 Vim 中按 i 进入插入模式并录入 FizzBuzz 程序, 按 Esc 回到正常模式, 再在命令行模式输入 :w。图 32 记录了执行保存前的状态, 按下回车后文件写入完成。

```
def fizz_buzz(limit):
    for i in range(limit):
        if i % 3 == 0:
            print("fizz", end="")
        if i % 5 == 0:
            print("fizz", end="")
        if i % 3 and i % 5:
            print(i, end="")
        print()

def main():
    fizz_buzz(20)

~
fizz_buzz.py[+] [unix] (07:59 01/01/1970)
:w
```

图 32 在 Vim 中录入并保存 FizzBuzz 程序

9. 使用搜索和移动命令修改代码。先用 /range 搜索到循环范围, 再用 2w 移到 limit, 配合 i、e 和 a 在两侧补入边界, 最终得到 range(1, limit + 1), 见图 33。

10. 使用文本对象修改字符串。跳转到 5 的倍数所在分支后, 用 f" 找到引号, 再以 ci" 一次替换引号内部内容, 将错误的 fizz 改为 buzz, 见图 34。


```
def fizz_buzz(limit):
    for i in range(1, limit + 1):
        if i % 3 == 0:
            print("fizz", end="")
        if i % 5 == 0:
            print("fizz", end="")
        if i % 3 and i % 5:
            print(i, end="")
        print()
fizz_buzz.py[+] [unix] (17:10 02/09/2026)
:w
```

图 33 使用 Vim 搜索与移动命令修改循环范围

```
for i in range(1, limit + 1):
    if i % 3 == 0:
        print("fizz", end="")
    if i % 5 == 0:
        print("buzz", end="")
    if i % 3 and i % 5:
        print(i, end="")
    print()
fizz_buzz.py [unix] (17:19 02/09/2026)
```

图 34 使用 Vim 文本对象替换字符串

2.3 Debugging and Profiling

这一部分先围绕错误的平均值函数完成调试，再围绕队列实现完成性能分析。前四项按“复现问题、观察异常、定位原因、修复验证”的顺序进行；确认程序正确后，再从单次测量、重复基准测试、函数级分析和规模变化几个角度比较性能。

Debugging

1. 复现错误并查看调试日志。原程序计算 [10,20,30,40] 的平均值时输出 33.33，而逐项写入的 DEBUG 日志表明累计和为 100，因此求和过程正常，问题更可能出在最后的除法，见图 35。

```
xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第二周/课后/debugging_and_profiling (main)
$ python stats_bug.py
mean=33.33

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第二周/课后/debugging_and_profiling (main)
$ rm -f debug.log
$ python stats_bug.py
$ cat debug.log
mean=33.33
DEBUG:value=10 total=10
DEBUG:value=20 total=30
DEBUG:value=30 total=60
DEBUG:value=40 total=100
```

图 35 错误结果及逐项累加日志

2. 使用边界输入观察异常栈。把输入缩小为单元素列表 [10] 后，原返回语句把 `len(values)` 减 1 后作为分母，此时分母变为 0，程序触发除零异常；异常栈直接指向平均值函数第 11 行，见图 36。

```
xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第二周/课后/debugging_and_profiling (main)
$ python -c "from stats_bug import mean; print(mean([10]))"
Traceback (most recent call last):
  File "<string>", line 1, in <module>
  File "D:\desktop\系统开发工具基础\第二周\课后\debugging_and_profiling\stats_bug.py", line 11, in mean
    return total / (len(values) - 1)
ZeroDivisionError: division by zero
```

图 36 单元素输入触发的 ZeroDivisionError

3. 使用 pdb 检查变量并定位错误。在返回语句设置断点后，pdb 显示总和为 100、列表长度为 4；正确结果应为 $100/4 = 25$ ，而原表达式实际使用分母 3；这些运行时数据确认错误来自多减的 1，见图 37。

```
xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第二周/课后/debugging_and_profiling (main)
$ python -m pdb stats_bug.py
> d:\desktop\系统开发工具基础\第二周\课后\debugging_and_profiling\stats_bug.py(1)<module>()
-> import logging
(Pdb) b stats_bug.py:11
Breakpoint 1 at d:\desktop\系统开发工具基础\第二周\课后\debugging_and_profiling\stats_bug.py:11
(Pdb) r
> d:\desktop\系统开发工具基础\第二周\课后\debugging_and_profiling\stats_bug.py(11)mean()
-> return total / (len(values) - 1)
(Pdb) p values
[10, 20, 30, 40]
(Pdb) p total
100
(Pdb) p len(values)
4
(Pdb) p len(values) - 1
3
(Pdb) p total / len(values)
25.0
(Pdb) q
```

图 37 pdb 断点处的变量检查结果

4. 修复平均值错误并验证结果。将返回语句改为 `total/len(values)` 后重新测试，四个数的平均值变为 25.00，单元素列表也能正确返回 10.0，完成了从复现、定位到修复的闭环，见图 38；修复后的程序见代码 10。

```
xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第二周/课后/debugging_and_profiling (main)
$ python stats_bug.py
python -c "from stats_bug import mean; print(mean([10]))"
mean=25.00
10.0
```

图 38 修复后普通输入与边界输入均得到正确结果

```
1 import logging
2
3 logging.basicConfig(filename="debug.log", level=logging.DEBUG, format="%(
    levelname)s:%(message)s")
4
5
```

```
6 def mean(values):
7     total = 0
8     for value in values:
9         total += value
10        logging.debug("value=%s total=%s", value, total)
11    return total / len(values)
12
13
14 def report(values):
15     return f"mean={mean(values):.2f}"
16
17
18 if __name__ == "__main__":
19     print(report([10, 20, 30, 40]))
```

代码 10 修复后的平均值程序

Profiling

后续实例比较两种队列：list_queue 每次删除列表首元素，deque_queue 则调用双端队列的 popleft。用于计时与内存测量的完整程序见代码 11。

```
1 import argparse
2 import time
3 import tracemalloc
4 from collections import deque
5
6
7 def list_queue(n):
8     queue = list(range(n))
9     total = 0
10    while queue:
11        total += queue.pop(0)
12    return total
13
14
15 def deque_queue(n):
16     queue = deque(range(n))
17     total = 0
18    while queue:
19        total += queue.popleft()
```

```
20     return total
21
22
23 def main():
24     parser = argparse.ArgumentParser()
25     parser.add_argument("mode", choices=["list", "deque"])
26     parser.add_argument("n", type=int)
27     args = parser.parse_args()
28
29     function = list_queue if args.mode == "list" else deque_queue
30     tracemalloc.start()
31     start = time.perf_counter()
32     result = function(args.n)
33     elapsed = time.perf_counter() - start
34     _, peak = tracemalloc.get_traced_memory()
35     tracemalloc.stop()
36
37     print(
38         f"mode={args.mode} result={result} "
39         f"elapsed={elapsed:.6f} peak_kib={peak / 1024:.1f}"
40     )
41
42
43 if __name__ == "__main__":
44     main()
```

代码 11 list 与 deque 队列性能测试程序

5. 比较运行时间与峰值内存。两种实现处理 50000 个元素后都得到 1249975000，说明功能一致；本次测量中 list 用时 0.327324 s，deque 用时 0.085437 s，峰值内存都约为 1.9 MiB，见图 39。

```
xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第二周/课后/debugging_and_profiling (main)
● $ for mode in list deque; do
    python queue_bench.py "$mode" 50000
done
mode=list result=1249975000 elapsed=0.327324 peak_kib=1945.3
mode=deque result=1249975000 elapsed=0.085437 peak_kib=1958.0
```

图 39 两种队列的单次时间与峰值内存

6. 使用 timeit 重复测量。为减小单次计时的偶然误差，分别进行 5 轮、每轮 3 次测量；list 每次约 233 ms，deque 每次约 4.28 ms，前者约为后者的 54 倍，见图 40。

7. 使用 cProfile 分析 list 热点。将分析数据保存为 list.prof 后用 pstats 按内部耗时排序；50000 次 list.pop 共占 0.233 s，是 list 版本最明显的热点，见图 41。

```
xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础/第二周/课后/debugging_and_profiling (main)
$ python -m timeit -r 5 -n 3 -s "from queue_bench import list_queue" "list_queue(50000)"
python -m timeit -r 5 -n 3 -s "from queue_bench import deque_queue" "deque_queue(50000)"
3 loops, best of 5: 233 msec per loop
3 loops, best of 5: 4.28 msec per loop
```

图 40 timeit 重复测量 list 与 deque

```
xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础/第二周/课后/debugging_and_profiling (main)
$ python -m cProfile -o list.prof queue_bench.py list 50000
python -c "import pstats; pstats.Stats('list.prof').strip_dirs().sort_stats('tottime').print_stats(10)"
mode=list result=1249975000 elapsed=0.323726 peak_kib=1946.3
Wed Sep 2 19:49:10 2026 list.prof

60615 function calls (60458 primitive calls) in 0.351 seconds

Ordered by: internal time
List reduced from 345 to 10 due to restriction <10>

ncalls  tottime  percall  cumtime  percall filename:lineno(function)
50000   0.233    0.000    0.233    0.000 {method 'pop' of 'list' objects}
1       0.001    0.001    0.324    0.324 queue_bench.py:7(list_queue)
100     0.004    0.000    0.004    0.000 {built-in method nt.stat}
10      0.003    0.000    0.003    0.000 {built-in method io.open_code}
10      0.003    0.000    0.003    0.000 {built-in method marshal.loads}
2       0.002    0.001    0.002    0.001 {built-in method _imp.create_dynamic}
324     0.002    0.000    0.003    0.000 <frozen importlib._bootstrap_external>:96(_path_join)
58/56   0.001    0.000    0.001    0.000 {built-in method builtins.__build_class__}
10      0.001    0.000    0.001    0.000 {method 'read' of '_io.BufferedReader' objects}
66      0.001    0.000    0.008    0.000 <frozen importlib._bootstrap_external>:1604(find_spec)
```

图 41 list 版本的 cProfile 统计

8. 使用 cProfile 分析 deque 热点。以相同参数分析 deque 版本时，50000 次 popleft 只占 0.006 s，整个分析过程也由 list 的 0.351 s 降至 0.117 s，两份报告的差异见图 42。

```
xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础/第二周/课后/debugging_and_profiling (main)
$ python -m cProfile -o deque.prof queue_bench.py deque 50000
python -c "import pstats; pstats.Stats('deque.prof').strip_dirs().sort_stats('tottime').print_stats(10)"
mode=deque result=1249975000 elapsed=0.090058 peak_kib=1959.1
Wed Sep 2 19:50:03 2026 deque.prof

60615 function calls (60458 primitive calls) in 0.117 seconds

Ordered by: internal time
List reduced from 345 to 10 due to restriction <10>

ncalls  tottime  percall  cumtime  percall filename:lineno(function)
1       0.084    0.084    0.090    0.090 queue_bench.py:15(deque_queue)
50000   0.006    0.000    0.006    0.000 {method 'popleft' of 'collections.deque' objects}
100     0.004    0.000    0.004    0.000 {built-in method nt.stat}
10      0.003    0.000    0.003    0.000 {built-in method io.open_code}
10      0.003    0.000    0.003    0.000 {built-in method marshal.loads}
324     0.002    0.000    0.003    0.000 <frozen importlib._bootstrap_external>:96(_path_join)
2       0.002    0.001    0.002    0.001 {built-in method _imp.create_dynamic}
58/56   0.001    0.000    0.001    0.000 {built-in method builtins.__build_class__}
10      0.001    0.000    0.001    0.000 {method 'read' of '_io.BufferedReader' objects}
66      0.001    0.000    0.007    0.000 <frozen importlib._bootstrap_external>:1604(find_spec)
```

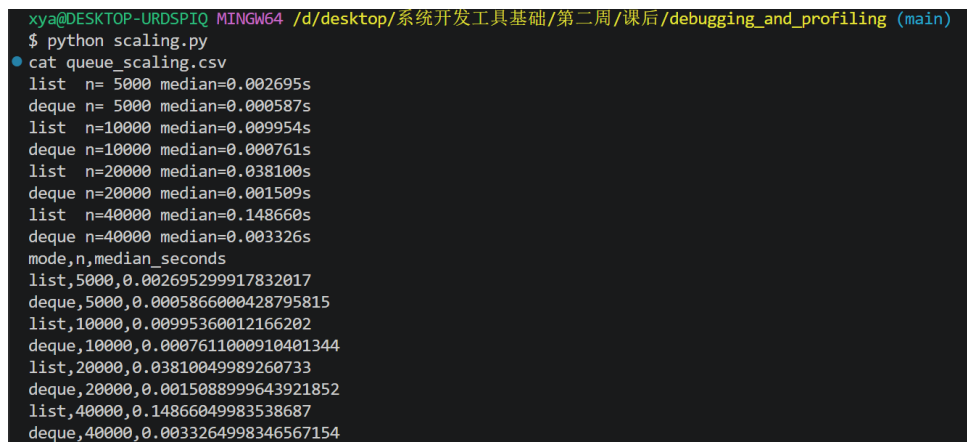
图 42 deque 版本的 cProfile 统计

9. 测量不同数据规模下的性能。为了观察增长趋势，使用代码 12 对 5000、10000、20000、40000 四种规模各重复测量 5 次并取中位数，再把结果写入 CSV；数据规模每翻倍，list 用时接近增长 4 倍，而 deque 的增长更接近线性，见图 43。

```
1 import csv
2 import statistics
3 import timeit
4
5 from queue_bench import deque_queue, list_queue
```

```
6
7 sizes = [5000, 10000, 20000, 40000]
8 methods = [("list", list_queue), ("deque", deque_queue)]
9
10 with open("queue_scaling.csv", "w", newline="") as file:
11     writer = csv.writer(file)
12     writer.writerow(["mode", "n", "median_seconds"])
13
14     for n in sizes:
15         for name, function in methods:
16             times = timeit.repeat(lambda: function(n), number=1, repeat=5)
17             median = statistics.median(times)
18             writer.writerow([name, n, median])
19             print(f"{name:5} n={n:5} median={median:.6f}s")
```

代码 12 多种数据规模的重复计时程序



```
xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第二周/课后/debugging_and_profiling (main)
$ python scaling.py
cat queue_scaling.csv
list n= 5000 median=0.002695s
deque n= 5000 median=0.000587s
list n=10000 median=0.009954s
deque n=10000 median=0.000761s
list n=20000 median=0.038100s
deque n=20000 median=0.001509s
list n=40000 median=0.148660s
deque n=40000 median=0.003326s
mode,n,median_seconds
list,5000,0.002695299917832017
deque,5000,0.0005866000428795815
list,10000,0.00995360012166202
deque,10000,0.0007611000910401344
list,20000,0.03810049989260733
deque,20000,0.0015088999643921852
list,40000,0.14866049983538687
deque,40000,0.0033264998346567154
```

图 43 不同队列规模对应的计时数据

10. **绘制性能变化曲线。**读取上一步生成的 `queue_scaling.csv`，用 Matplotlib 将两组中位数绘制为折线；`list` 曲线随规模快速上升，而 `deque` 始终贴近横轴，图形与前面的计时和热点分析一致，见图 44。

11. **查找占用端口的进程并释放端口。**启动 4444 端口上的临时 HTTP 服务后，`netstat -ano` 找到监听进程 PID 55428；停止服务并再次查询时已无监听记录，终端输出 `port 4444 released`，见图 45。

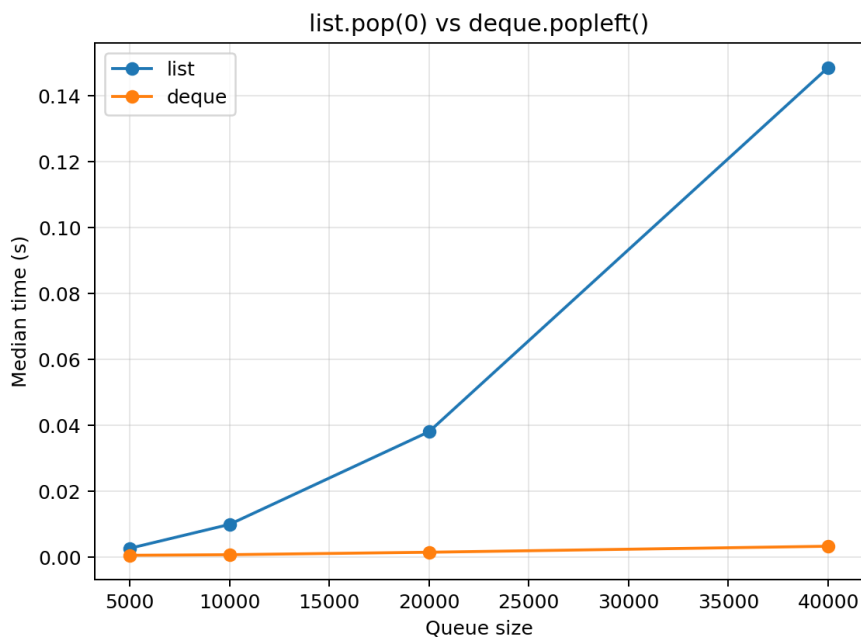


图 44 list.pop(0) 与 deque.popleft() 的性能曲线

```
xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础 (main)
$ netstat -ano | grep ':4444'
TCP    0.0.0.0:4444      0.0.0.0:0        LISTENING        55428
TCP    [::]:4444        [::]:0           LISTENING        55428

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础 (main)
$ netstat -ano | grep ':4444' || echo "port 4444 released"
port 4444 released

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第二周/课后/debugging_and_profiling (main)
$ python -m http.server 4444
Serving HTTP on :: port 4444 (http://[::]:4444/) ...

Keyboard interrupt received, exiting.

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第二周/课后/debugging_and_profiling (main)
$
```

```
Microsoft Windows [版本 10.0.26200.9168]
(c) Microsoft Corporation. 保留所有权利。

D:\desktop\系统开发工具基础\第二周\课后\debugging_and_profiling>powershell -NoProfile -Command "Get-Process -Id (Get-NetTCPConnection -LocalPort 4444).OwningProcess"

Handles NPM(K) PM(K) WS(K) CPU(s) Id SI ProcessName
-----
155 15 13116 22672 0.17 55428 2 python
```

图 45 定位端口监听进程并确认端口释放

3 解题感悟

先记录一下这周学会了什么吧。命令行环境这部分以前接触得比较零散，之前大概会些常用的文本操作和一些简单的脚本命令吧，这次好好巩固了第一周的 shell 和各种脚本语言，延申了进程、ssh 等知识点，明显感觉到 shell 比之前想的要厉害得多，比如说控制命令之间怎么传数据、程序在什么环境里运行，以及后台任务什么时候结束等。有些符号现在还是容易混，之后真用到估计还是得再查一下。

开发环境这部分让我重新认识了一下平时一直在用的 VS Code。之前的话写代码其实写的都比较简单没好好了解 IDE 的功能，更多是把补全和报错提示当成编辑器自带的功能，这次才知道解释器、Python 扩展和 Pylance 各自在做什么。中间还装了 Ruff 和 Error Lens 试了一下，Error Lens 把提示直接铺在代码后面看得太烦了，所以实验完成后还是卸载了。Vim 刚开始用挺不习惯的，有命令要记得都挺烦的，熟练以后应该会比一直移动光

标快很多。

这周的 Debugging 和 Profiling 部分让我更熟悉了调试方式吧，对性能的分析也更加重视了。之前打断点调试不是很熟练，这次练习了两种调试方式；性能分析也让我发现“感觉哪个写法快”其实不太可靠，list 和 deque 在数据量小时看起来区别不大，但用 timeit 重复测量、再看 cProfile 的热点和不同规模下的曲线，差距就很明显了。以后再碰到程序慢的问题，应该先测一下到底慢在哪里，而不是上来就凭感觉改一堆代码。

现在遇到报错我还是习惯性想问 ai，不过之后至少应该先自己看看异常栈、变量值和性能数据，知道问题大概出在哪，再让 ai 帮我解释或者找其它可能。

第二周课上实验、课后练习及实验报告均已提交至 Git 仓库，相关提交记录见图 46。

```
PS D:\desktop\系统开发工具基础> git log -6 --oneline --graph
* f020b61 (HEAD -> main, origin/main, origin/HEAD) 第二周实验报告基本完成，待补充commit截图
* 5a7911f 完成第二周课后练习，准备截图等
* 1cc318c 第二周课上q08任务完成
* 45a9e76 第二周课上q07任务完成
* 9b9d2a3 第二周课上q06任务完成
* 83539d1 第二周课上q05任务完成
```

图 46 第二周实验及报告的 Git 提交记录